

CSE 4241: Biomedical Engineering

Predicting Protein Secondary Structure

A Convolutional Recurrent Network on CB6133-Filtered

Submitted By

Prova Rani Paul

Roll: 2007037



Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

Abstract

Secondary structure prediction labels each residue of a protein as part of a Helix (H), Sheet (E), or Coil (C) region. This report builds a sequence-only classifier for the three-way (Q3) task: an embedding layer feeds a 1D convolution (for local motifs) into a bidirectional LSTM (for longer-range context), trained on a CB6133-filtered split of 5,600 training / 272 validation / 256 test protein sequences. With dropout, AdamW weight decay, learning-rate scheduling, and early stopping on validation loss, the model reaches **71.09%** per-residue accuracy on the held-out test set after 14 training epochs (best checkpoint at epoch 7).

1 Introduction

A protein’s secondary structure – whether a given residue sits in an alpha helix, a beta strand, or a more irregular coil/loop region – sits between its raw amino-acid sequence and its full three-dimensional fold. Because it is far cheaper to predict than tertiary structure but still informative for it, secondary structure prediction is a common intermediate step in structural bioinformatics pipelines, function annotation, and early-stage drug target analysis.

This report treats Q3 prediction as per-residue, 3-class sequence labeling, using only the amino-acid sequence as input (no evolutionary profile features such as Position-Specific Scoring Matrices, PSSMs, or Multiple Sequence Alignments, MSAs). The aim is a small, fully reproducible pipeline – data loading, label conversion, a CNN+BiLSTM model, and a regularized training loop – and an honest look at how far a sequence-only model gets on a CB6133-filtered split before profile information is introduced.

2 Data and Preprocessing

2.1 Dataset

The experiments use a CB6133-filtered split of **5,600 training**, **272 validation**, and **256 test** protein sequences [1], each paired with a per-residue DSSP secondary-structure string of the same length. Sequence lengths in the training set range from 12 to 696 residues (mean 208.6); none exceed the 700-residue budget used for padding, so no sequence is truncated.

2.2 From 8-state DSSP labels to Q3

Raw secondary-structure labels follow the 8-state **DSSP** scheme (Define Secondary Structure of Proteins) [2], which is collapsed to the standard 3-class Q3 scheme before training:

DSSP labels	Q3 class
H, G, I	Helix (H)
E, B	Sheet (E)
T, S, L, C, -, . (everything else)	Coil (C)

The conversion is a simple character-level remap and is idempotent – applying it to a string that is already H/E/C changes nothing – so the same code path works regardless

of which label format a given split ships with. On the training set this produces a class split of **38.5% Helix**, **22.8% Sheet**, and **38.8% Coil** at the residue level, i.e. Sheet is the clear minority class.

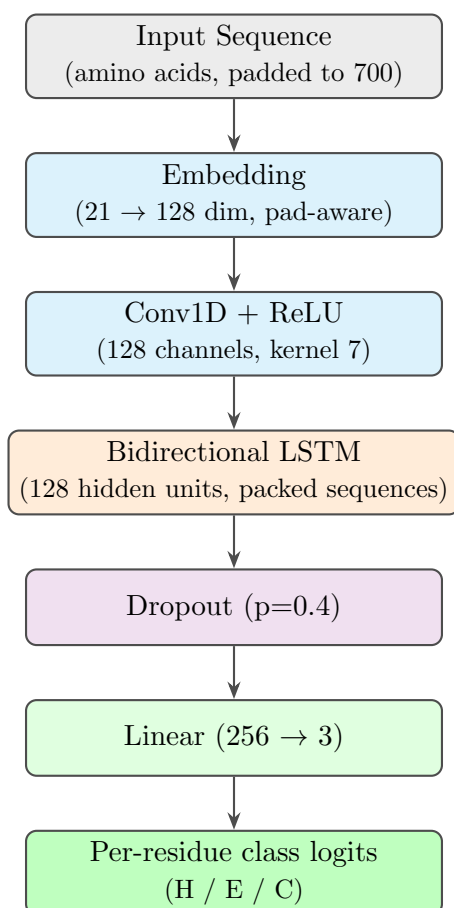
2.3 Encoding and Padding

Each of the 20 standard amino acids is mapped to an integer 1–20, with index 0 reserved for padding and for any unrecognized symbol (e.g. X). Sequences are padded or truncated to a fixed length of 700. Label positions corresponding to padding are set to a sentinel value of -100 , which PyTorch’s `CrossEntropyLoss` ignores by construction; the same mask is reused for the accuracy metric, so padded positions never contribute to any reported loss or accuracy number, training or test.

Each example’s true, unpadded length is also stored and passed through to the model, so that the recurrent layer (Section 3) can be told exactly where the real sequence ends.

3 Model Architecture

The classifier is a four-stage PyTorch model:



The full model has **382,467** trainable parameters.

Why this combination. The embedding turns discrete residue identities into a dense representation the rest of the network can work with. The convolution, with a kernel width of 7, looks at a local window of ± 3 neighboring residues – enough to pick up

short motifs that correlate with local helix/sheet propensity. A single-layer bidirectional LSTM then scans the convolved features in both directions, which is important because secondary structure depends on context well outside any fixed local window (most clearly for beta strands, which pair with other, potentially distant, strands). A dropout layer and a final linear layer turn the recurrent output into per-residue logits over the three classes; softmax is applied implicitly inside the training loss and explicitly (via `argmax`) at inference time.

Packed sequences in the LSTM. Feeding a zero-padded batch directly into an LSTM is wasteful and, for the backward direction, actively harmful: it would have to step through a long run of padding tokens before reaching any real residues, polluting the hidden state it carries into the genuine part of the sequence. Wrapping the recurrent layer with `pack_padded_sequence/pad_packed_sequence` and each sample’s true length avoids this entirely – the LSTM only ever processes real residues. Given that the average sequence here is only 208.6 residues against a 700-residue padding budget, this matters for the large majority of training examples.

4 Results

4.1 Training Configuration

Table 1 lists the configuration used for the reported run. Optimization used `AdamW` [3] (Adam with decoupled weight decay), a `ReduceLROnPlateau` schedule tied to validation loss, and early stopping with restoration of the best-validation-loss checkpoint – so the numbers below describe the best epoch seen during training, not whatever epoch happened to run last. Inverse-frequency class weights were computed from the training distribution but left *inactive* for this run (standard, unweighted cross-entropy was used); Section 5 revisits this choice.

Table 1: Training configuration.

Parameter	Value	Parameter	Value
Max sequence length	700	Batch size	64
Embedding dim.	128	Learning rate	1×10^{-3}
Conv1D channels	128	Weight decay	1×10^{-4}
Kernel size	7	LR patience / factor	3 / 0.5
BiLSTM hidden size	128	Early-stop patience	7 epochs
BiLSTM layers	1	Class-weighted loss	Computed, disabled
Dropout	0.4	Max epoch budget	25

Training stopped early after **14** epochs (no validation-loss improvement for 7 consecutive epochs), with the best checkpoint at **epoch 7** (validation loss 0.6787, validation accuracy 70.68%).

4.2 Training Curves

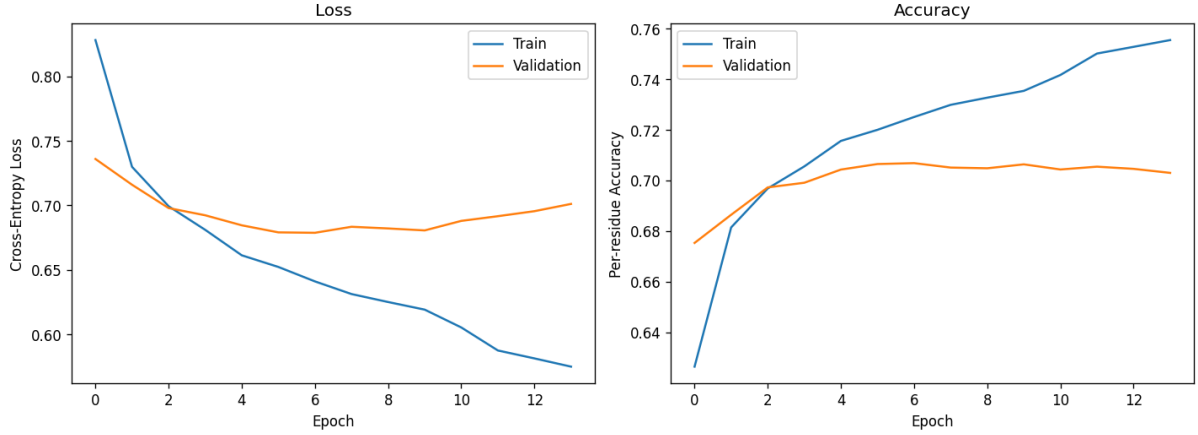


Figure 1: Training and validation loss (left) and per-residue accuracy (right) over the 14 epochs that were run. Validation loss bottoms out around epoch 6–7 and then climbs slowly while training loss keeps falling – the standard signature of the model starting to memorize the training set, which is exactly what the early-stopping/best-checkpoint strategy is designed to catch.

4.3 Weight Evolution

The training loop saved a heat-map of the output layer’s weight matrix after every epoch (all 14 are combined into the supplementary animation `weight_updates.gif`). Rather than reproduce a handful of arbitrarily chosen frames, Figure 2 shows six epochs spaced across the full run – the random initialization (epoch 1), two points during active learning (epochs 3 and 5), the best early-stopping checkpoint (epoch 7, outlined in red), and two post-checkpoint epochs (10 and 14) where the validation loss has already started to rise. This captures the full before/best/after story in one figure without the redundancy of near-identical adjacent epochs. The complete 14-epoch grid is included as Figure 4 in the appendix for anyone who wants epoch-by-epoch detail.

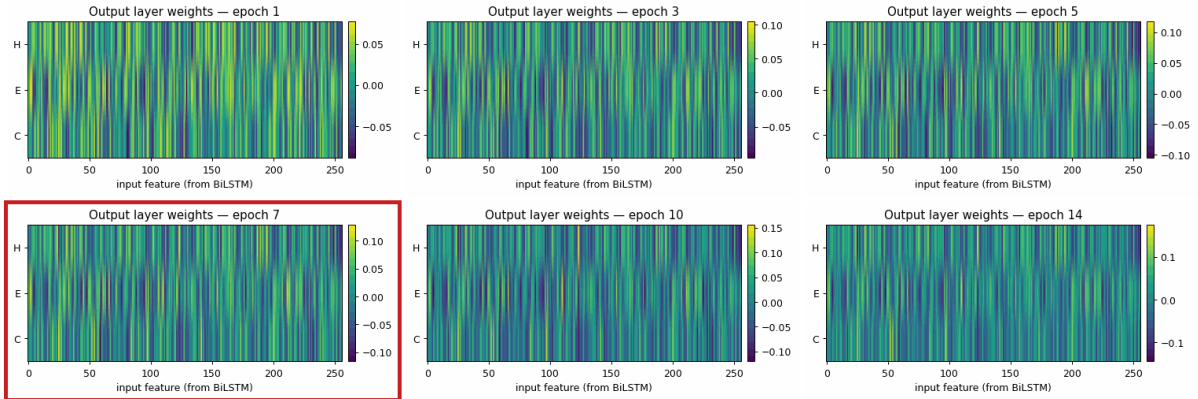


Figure 2: Output-layer weight matrix at epochs 1, 3, 5, 7, 10, and 14. The epoch-7 panel (red outline) is the best-validation-loss checkpoint that was restored for final evaluation. Rows are the three output classes (H/E/C); columns are the 256 BiLSTM output features feeding the final linear layer. The pattern visibly sharpens through epoch 7 and then continues to drift afterward as the model overfits.

4.4 Test Set Evaluation

Table 2: Per-class and aggregate evaluation metrics on the held-out test set.

Class	Precision	Recall	F1-score	Support
Helix (H)	0.78	0.75	0.76	19,675
Sheet (E)	0.68	0.61	0.64	12,464
Coil (C)	0.67	0.74	0.70	20,580
Weighted average	0.71	0.71	0.71	52,719
Macro average	0.71	0.70	0.70	52,719
Overall accuracy: 71.09%				

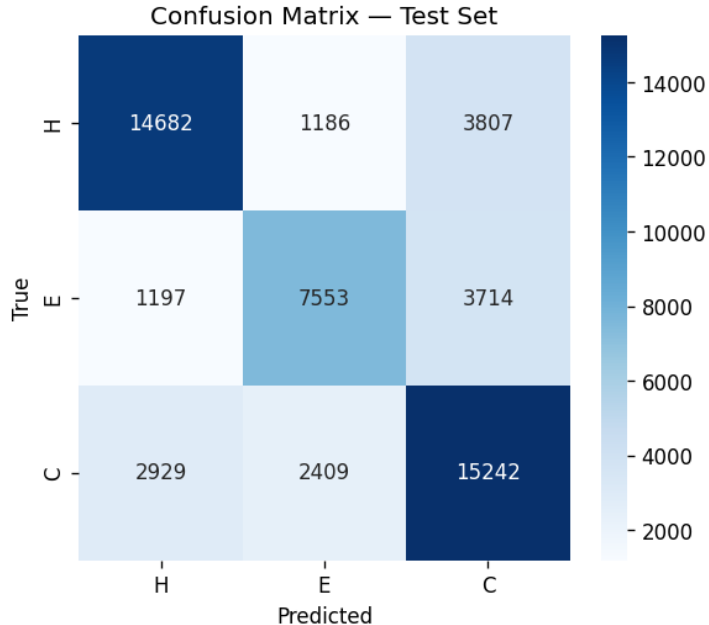


Figure 3: Confusion matrix on the test set (residue counts, $n = 52,719$). The diagonal dominates each row, but Sheet is visibly the weakest class: 3,714 true Sheet residues are predicted Coil and 1,197 predicted Helix, against only 7,553 predicted correctly. Coil and Helix are confused with each other and with Sheet roughly symmetrically, consistent with DSSP assigning gradual, sometimes ambiguous labels at class boundaries.

5 Discussion

Overall performance. 71.09% per-residue accuracy is comfortably above the roughly 39% a majority-class baseline would get on this label distribution, and sits inside the 65–75% range typically reported for sequence-only Q3 models that do not use evolutionary profile features. Sheet (E) is the weakest class by a clear margin (F1 = 0.64 versus 0.76 and 0.70 for Helix and Coil) – it is both the rarest class in training (22.8% of residues) and the most structurally non-local of the three, since beta strands depend on pairing with other strands that may be far away in sequence, something a purely sequential model can only partly capture.

Effect of leaving class weighting off. The notebook computes inverse-frequency class weights for use inside `CrossEntropyLoss`, but this run leaves that flag off, so the loss treats all three classes equally despite Sheet’s lower frequency. That choice plausibly explains part of Sheet’s weak recall (0.61): with unweighted loss, the optimizer has less incentive to fix Sheet mistakes relative to the more common Helix and Coil classes. Enabling `use_class_weights` is a natural next experiment, likely trading a small amount of overall accuracy for a more balanced confusion matrix.

Overfitting and regularization. Figure 1 shows validation loss turning upward by around epoch 8 while training loss keeps falling through epoch 14, with the train/validation accuracy gap widening from under one point near the best epoch to roughly five points by the time training stops. Dropout (0.4), AdamW weight decay (10^{-4}), and best-checkpoint early stopping are the levers already in place to manage this; given the gap that still appears, a somewhat smaller model (fewer LSTM hidden units, or staying at a single LSTM layer rather than adding a second) or stronger regularization would be reasonable next steps before reaching for more data.

Why packed sequences mattered here. With a mean sequence length of 208.6 against a 700-residue pad budget, most training examples are more than two-thirds padding. Packing the sequences before the BiLSTM keeps that padding from leaking into the learned representations, which is especially important for the backward LSTM direction, as discussed in Section 3.

Limitations. The model uses no evolutionary information (PSSM or MSA-derived profiles), which is consistently the single biggest source of accuracy gains in the secondary-structure literature. Fixed 700-residue padding is also a coarse choice given how short most sequences in this split actually are; packing removes its effect on the recurrent layer, but it still means a sizable share of every batch’s compute is spent on padding tokens that carry no information.

6 Possible Next Steps

- **Turn on class weighting.** Now that Sheet’s weaker recall is visible in the test results, enabling the already-computed inverse-frequency weights is the cheapest experiment to try next.
- **Evolutionary features.** Adding PSSM or MSA-derived profiles as extra per-residue input channels remains the most reliable lever for closing the gap to higher Q3 accuracy.
- **More LSTM capacity, carefully.** The config file already flags `lstm_layers` as a knob to revisit “once overfitting is under control” – a second layer is worth trying alongside slightly stronger regularization, given the train/validation gap observed here.
- **Attention or transformer-based encoders.** Self-attention can model long-range pairing (e.g. beta-sheet partners) more directly than a recurrent layer has to.
- **Pretrained protein language models.** Frozen or fine-tuned embeddings from models such as ESM [4] or ProtBERT [5] have repeatedly been shown to improve downstream structure-prediction accuracy.

7 Conclusion

This project implements an end-to-end pipeline for Q3 secondary structure prediction: DSSP-to-Q3 label conversion, length-aware encoding and padding, a compact CNN+BiLSTM model (382K parameters), and a training loop with dropout, weight decay, learning-rate scheduling, and early stopping. On a CB6133-filtered split of 256 held-out test sequences, the model reaches 71.09% per-residue accuracy, in line with what sequence-only models of this kind typically achieve. The training curves and per-class metrics point to two concrete next steps that would likely help most: turning on the already-computed class weights to address Sheet’s weaker recall, and adding evolutionary profile features to close the larger remaining gap.

References

- [1] J. Zhou and O. G. Troyanskaya, “Deep supervised and convolutional generative stochastic network for protein secondary structure prediction,” in *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014, pp. 745–753.
- [2] W. Kabsch and C. Sander, “Dictionary of protein secondary structure: Pattern recognition of hydrogen-bonded and geometrical features,” *Biopolymers*, vol. 22, no. 12, pp. 2577–2637, 1983.
- [3] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [4] A. Rives, J. Meier, T. Sercu, et al., “Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences,” *Proceedings of the National Academy of Sciences*, vol. 118, no. 15, e2016239118, 2021.
- [5] A. Elnaggar, M. Heinzinger, C. Dallago, et al., “ProtTrans: Towards cracking the language of life’s code through self-supervised deep learning and high performance computing,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.

Appendix: Full Weight-Evolution Sequence

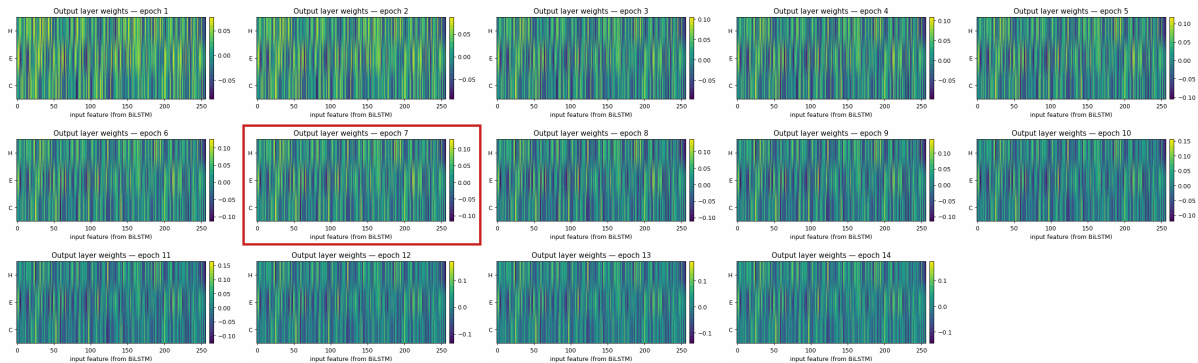


Figure 4: All 14 saved epochs of the output-layer weight matrix, for reference. Epoch 7 (red outline) is the restored best checkpoint. The same sequence is also provided as an animated GIF (`weight_updates.gif`) alongside this report.